

MLOps Face Recognition System

Low-Level Design Document

Document Type: Design & Implementation Review — LLD

System: Siamese Face Recognition — Full MLOps Pipeline

Components: Flask API · Airflow DAG · Trainer · MLflow · Prometheus/Grafana · Docker Compose

Technology: PyTorch · Optuna · Airflow 3.1.8 · MLflow 2.11.1 · Docker Compose

April 2026

1 Introduction & Scope

This document presents the **Low-Level Design (LLD)** for the MLOps Face Recognition system. Where the HLD document describes the three-plane architecture at the conceptual level, this document descends into the concrete implementation: every Docker Compose service, its Dockerfile, its exposed ports, its internal modules, its inter-service communication paths, and the complete REST API endpoint contract. Together, these two documents provide the full design specification required for the implementation review.

All six services share a single Docker bridge network named **mlops-net**. Services resolve each other by DNS hostname (e.g., mlflow, airflow-webserver, prometheus) rather than by IP address, ensuring that container restarts do not break inter-service connectivity. Volumes are declared at the Compose level and mounted into the appropriate containers, providing persistent storage across container lifecycles.

2 Low-Level Design Diagram — Docker Compose Architecture

Figure 2 below maps every Docker Compose service to its Dockerfile, exposed ports, internal modules, and inter-service communication paths. The diagram is colour-coded by functional group: **blue** for Inference (Flask), **grey/dark** for Training, **orange/brown** for MLflow, **teal/green** for Airflow and Observability, and **purple** for Infrastructure (databases, volumes). Read the arrows as data-flow or control-flow dependencies — solid arrows indicate synchronous REST calls, dashed arrows indicate asynchronous triggers or scrape relationships.

The LLD diagram below maps every Docker Compose service to its Dockerfile, exposed ports, internal modules, and inter-service communication paths. All services share the **mlops-net** bridge network and communicate via named DNS hostnames.

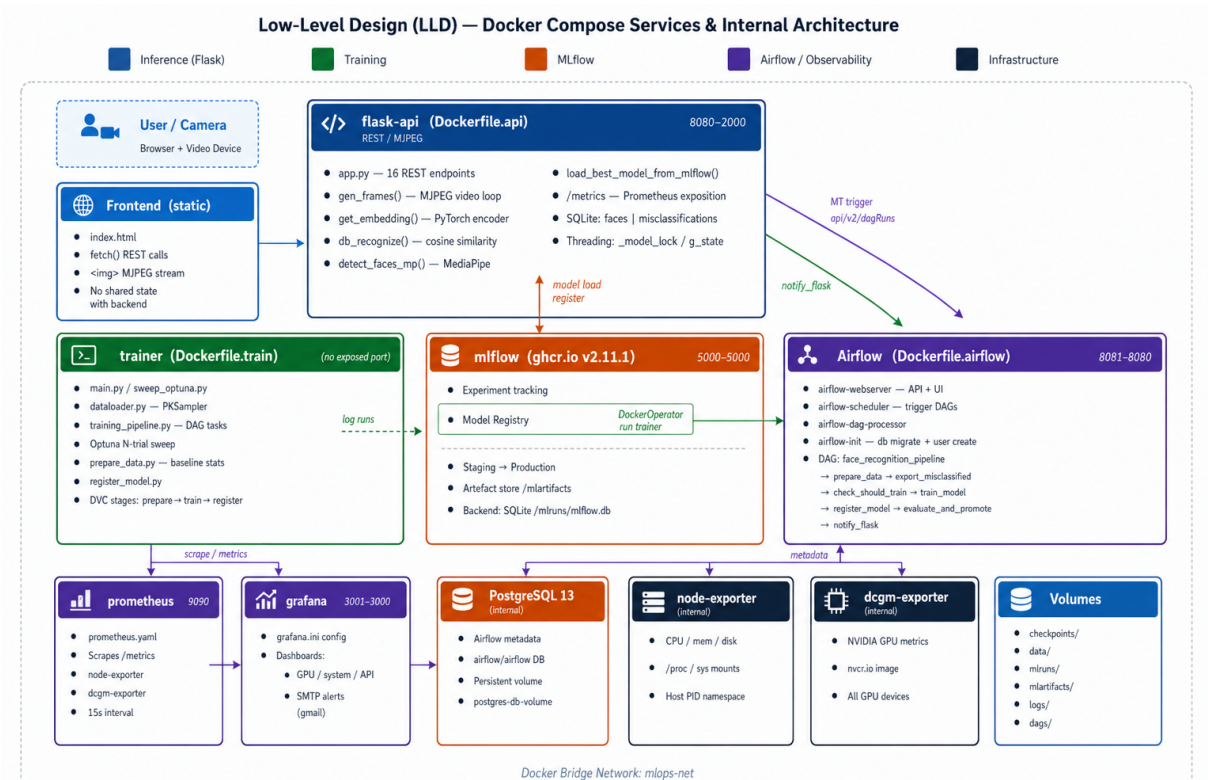


Figure 2 — Low-Level Design Diagram (LLD): Docker Compose services, Dockerfiles, ports, volumes, and data flows

2.5 Key Operational Behaviours

The following behaviours describe how the system operates end-to-end across registration, inference, re-training, and data lifecycle management.

Face Registration & Image Persistence

During registration, the user presents their face to the camera. The system captures face crops, computes embeddings, and stores them in the SQLite database. **Importantly, the raw captured images are also saved to disk** (under the person's directory) to support future fine-tuning rounds. This ensures that each registered identity contributes real-world appearance data to subsequent model improvement cycles.

Local Live Inference

Inference happens entirely locally, in real-time. The Flask API runs the full pipeline — MediaPipe face detection → ResNet50 embedding extraction → cosine similarity matching — on the host machine or local Docker environment. No frames or embeddings are transmitted to an external server during recognition, ensuring low latency and privacy.

Re-training Trigger Policy

Automated re-training is triggered by one of two conditions: (1) **Misclassification threshold breach** — when the number of HITL-reported misclassifications exceeds a configurable threshold, the Flask API authenticates with Airflow via JWT and triggers the face_recognition_pipeline DAG immediately; (2) **Weekly schedule** —

The diagram confirms that all six service groups share the **mlops-net** Docker bridge network (shown at the very bottom). Every inter-service call uses the container DNS hostname — no hardcoded IPs. The `notify_flask` arrow from Airflow back to the Inference Plane closes the self-improvement loop: after a new model is promoted to Production in MLflow, Airflow notifies Flask to hot-reload the weights and re-compute all stored face embeddings.

3 Docker Compose Services — Detailed Breakdown

This section provides an in-depth explanation of each Docker Compose service: its purpose, Dockerfile, network configuration, internal components, and communication with other services. Understanding each service in isolation — and then how they compose — is essential for maintenance, debugging, and future extension.

3.1 flask-api — Inference Service

flask-api

Dockerfile: Dockerfile.api | Port(s): 8080 (host) → 2000 (Internal)

The **flask-api** container is the system's primary user-facing service. It serves the browser-based frontend (`index.html`), streams live MJPEG video, exposes 16 REST endpoints for face management and HITL feedback, and runs the full real-time inference pipeline entirely in-process. It is the only service that directly interacts with the end user or camera hardware.

Key internal modules: `app.py` (16 endpoints), `gen_frames()` (MJPEG loop), `get_embedding()` (ResNet50 encoder), `db_recognize()` (cosine similarity), `detect_faces_mp()` (MediaPipe), `_model_lock` (thread safety), `g_state` machine (`mode/last_result/HITL`), `update_misclassification_metric()` (Prometheus gauge)

Internal Architecture

The Flask application is structured around a **global state machine** (`g_state`) that tracks the current operational mode — one of *idle*, *registering*, *verifying*, *identifying* — along with the last recognition result, HITL pending flag, and model version. A `threading.Lock` (`_model_lock`) guards all model-related state to ensure thread-safe concurrent access from the MJPEG streaming thread and the REST request threads.

The video pipeline runs as a Python generator (`gen_frames()`, ~80 lines) that continuously reads frames from the camera via OpenCV's `VideoCapture`, passes each frame to MediaPipe's `FaceDetection` for bounding-box extraction, crops and resizes the detected face, calls `get_embedding()` to produce an L2-normalised 512-dimensional vector, then queries `db_recognize()` for the closest identity in the SQLite embedding store. The annotated frame is JPEG-encoded and yielded as a multipart MJPEG chunk.

External Dependencies

The flask-api service reads the active model from the **MLflow Model Registry** at startup via `load_best_model_from_mlflow()`. It writes misclassification records to the local SQLite database and, when the threshold is breached, makes an authenticated HTTP POST to the **Airflow** webserver's REST API to trigger the DAG. The Prometheus **monitoring stack** scrapes the `/metrics` endpoint of this service every 15 seconds.

Volumes & Persistence

Volume / Path	Type	Purpose
<code>./data/</code>	Bind mount (host → container)	Stores raw registration images per enrolled identity; survives container restart
<code>./faces.db</code> (SQLite)	Bind mount	Face embeddings keyed by name; misclassification audit log
<code>mlartifacts/</code>	Shared volume with MLflow	Read-only access to model checkpoints registered by the training pipeline

3.2 trainer — Model Training Service

trainer

Dockerfile: `Dockerfile.train` | **Port(s):** No exposed port (internal only)

The **trainer** container encapsulates the entire model training lifecycle. It is not a long-running service — it is spun up by Airflow's `DockerOperator` for each training task and exits when training completes. It supports both local GPU execution (default) and remote GPU dispatch via SSH (when `RUN_REMOTE=true`).

Key internal modules: `main.py` (single-run training), `sweep_optuna.py` (N-trial Optuna sweep), `dataloader.py` (LFWIdentityDataset, LFWPairDataset, PKSampler), `training_pipeline.py` (DAG task callables), `prepare_data.py` (dataset splitting), `register_model.py` (MLflow push), `dvc.yaml` (pipeline stages), `params.yaml` (hyperparameter source of truth)

Training Entry Points

Two entry points exist depending on the Airflow DAG configuration. **main.py** executes a single training run using the hyperparameters defined in `params.yaml`, resolved through `resolve_params()` which merges config defaults → `params.yaml` → any sweep JSON override. **sweep_optuna.py** launches an Optuna TPE study for N trials, sampling the joint space of learning rate, margin, embedding dimension, and P×K batch composition. Each trial is logged as an independent MLflow run. The best trial by validation loss is selected for registration.

DVC Pipeline Stages

Stage	Script	deps (inputs)	outs (outputs)
prepare_data	prepare_data.py	Raw LFW + VGGFace2 directories	data/train/, data/val/ split manifests
train	sweep_optuna.py or main.py	data/train/, data/val/, params.yaml	checkpoints/best_model.pt, MLflow run ID
register	register_model.py	checkpoints/best_model.pt, best_run.json	MLflow model version in Staging

Table — DVC pipeline stages, inputs, and outputs

PKSampler — Batch Construction

Training uses a **PK sampler** strategy: each mini-batch is constructed by sampling P identities and K images per identity, yielding batches of size P×K. This is essential for triplet mining — within each batch, hard and semi-hard triplets are selected online by comparing all anchor-positive-negative combinations. The PKSampler class enforces that P×K is divisible by the configured batch size and raises AssertionError otherwise (tested in TestPKSampler).

3.3 mlflow-server — Experiment Tracking & Model Registry

mlflow-server

Dockerfile: ghcr.io/mlflow v2.11.1 (pre-built image) | **Port(s):** 5000 (internal + host)

The **MLflow server** is the central coordination layer between the training and inference planes. It is a long-running service that persists experiment metadata, model artefacts, and registry state across all training runs. It is the only service that both the trainer (write) and the flask-api (read) depend on directly.

Key internal modules: Experiment tracking UI (port 5000), Model Registry (Staging / Production stages), Artefact store (/mlartifacts volume), SQLite backend (/mlruns/mlflow.db)

Model Registry Workflow

The registry enforces a two-stage promotion workflow. After training completes, register_model.py pushes the best checkpoint and registers it in **Staging**. The Airflow evaluate_and_promote() task then compares the candidate model's validation metric against the current **Production** model's baseline. If the candidate improves the metric, it is promoted to Production; otherwise it remains in Staging for manual review. This gate prevents regressions from being automatically deployed to the inference service.

Artefact Storage

All model checkpoints, Optuna study artefacts, and training metrics are stored under /mlartifacts — a Docker named volume shared between the MLflow server and the flask-api container. This shared volume

allows the Flask API to load model weights directly from the filesystem path returned by MLflow without requiring a separate file transfer step. The SQLite backend (/mlruns/mlflow.db) stores all run metadata, parameter logs, and registry state.

3.4 Airflow — Pipeline Orchestration (Three Containers)

airflow-webserver + airflow-scheduler + airflow-dag-processor

Dockerfile: Dockerfile.airflow | Port(s): 8081 (host) – 8080

Apache Airflow 3.1.8 orchestrates the end-to-end training pipeline as a Directed Acyclic Graph (DAG) named `face_recognition_pipeline`. It runs as three coordinated containers sharing the same PostgreSQL metadata database: the **webserver** provides the UI and REST API; the **scheduler** evaluates DAG triggers and queues task instances; the **dag-processor** parses DAG files. The `DockerOperator` launches the trainer container for the training task.

Key internal modules: `airflow-webserver` (API + UI), `airflow-scheduler` (trigger evaluation), `airflow-dag-processor` (DAG parsing), `face_recognition_pipeline` DAG (7 tasks), `airflow-init` (db migrate + user create), PostgreSQL 13 metadata backend

DAG Task Graph — `face_recognition_pipeline`

Task	Operator	What It Does
<code>prepare_data</code>	<code>PythonOperator</code>	Calls <code>prepare_data.py</code> to split LFW + VGGFace2 into train/val sets; merges misclassified images into training set
<code>export_misclassified</code>	<code>PythonOperator</code>	Exports all pending misclassification records from SQLite to a CSV file for inclusion in the next training batch
<code>check_should_train</code>	<code>PythonOperator</code> / <code>ShortCircuitOperator</code>	Checks if at least one misclassification has been logged since the last run; short-circuits if not, avoiding unnecessary GPU compute
<code>train_model</code>	<code>DockerOperator</code>	Launches the trainer container with <code>sweep_optuna.py</code> or <code>main.py</code> ; logs all runs to MLflow; returns best run ID via XCom
<code>register_model</code>	<code>PythonOperator</code>	Calls <code>register_model.py</code> to push the best checkpoint to MLflow and register it in Staging
<code>evaluate_and_promote</code>	<code>PythonOperator</code>	Compares candidate model metric against current Production baseline; promotes to Production if improved; logs decision to Airflow logs
<code>notify_flask/reload</code>	<code>PythonOperator</code>	Sends HTTP POST to <code>flask-api /reload</code> endpoint; Flask hot-swaps the active model; also purges SQLite misclassification records and triggers full re-embedding

Table — Airflow DAG tasks, operators, and responsibilities

Trigger Mechanisms

The DAG supports two independent trigger mechanisms. The **scheduled trigger** uses a weekly cron expression configured in the DAG definition — Airflow's scheduler evaluates this interval and queues a DAG run automatically. The **event trigger** is fired by the Flask API when the misclassification threshold is exceeded: the API calls `_get_airflow_token()` to obtain a short-lived JWT from Airflow's auth endpoint, then POSTs to `/api/v1/dags/face_recognition_pipeline/dagRuns` with a JSON payload. Airflow authenticates the JWT, validates the request, and immediately queues a new DAG run.

3.5 Monitoring Stack — Prometheus + Grafana + Exporters

prometheus + grafana + node-exporter + dcgm-exporter

Dockerfile: Pre-built images (no custom Dockerfile) | **Port(s):** Prometheus: 9090 | Grafana: 3001

The observability stack provides complete visibility into the runtime behaviour of all services. Prometheus scrapes metrics endpoints at a 15-second interval, storing time-series data that Grafana visualises in preconfigured dashboards. SMTP-based alerting notifies operators of anomalous conditions such as recognition latency spikes or elevated error rates.

Key internal modules: prometheus.yml (scrape config), grafana.ini (config), GPU/system dashboards, node-exporter (CPU/mem/disk via `/proc` and `/sys` mounts), dcgm-exporter (NVIDIA GPU metrics via `nvc.io` image + all GPU devices)

Metric / Exporter	Source	What It Measures
recognition_latency_seconds (histogram)	flask-api /metrics	End-to-end latency from frame capture to identity decision; P50/P95/P99 percentiles
misclassification_count (gauge)	flask-api /metrics	Current count of pending HITL-reported misclassifications; resets to 0 after re-training
model_version (gauge)	flask-api /metrics	Currently loaded MLflow model version number
node-exporter metrics	Host OS (<code>/proc</code> , <code>/sys</code>)	CPU utilisation, memory usage, disk I/O, network throughput per host
dcgm-exporter metrics	NVIDIA DCGM	GPU utilisation, memory usage, temperature, power draw — one set per GPU device

Table — Key metrics exported by the monitoring stack

3.6 Databases & Persistent Storage

PostgreSQL 13 (Airflow metadata) + SQLite (faces + misclassifications)

Dockerfile: postgres:13 (pre-built) + embedded SQLite | **Port(s):** PostgreSQL: 5432 (internal only)

The system uses two database technologies for different persistence requirements. PostgreSQL 13 serves Airflow's metadata backend — storing DAG definitions, task instance state, XCom values, and connection/variable configurations. SQLite serves the Flask API for low-latency local reads of face embeddings and misclassification records, avoiding network overhead on the hot inference path.

Key internal modules: PostgreSQL: airflow metadata DB, persistent volume (postgres-db-volume) | SQLite: faces table (name, embedding blob), misclassifications table (true_name, predicted, score, timestamp)

Table / DB	Schema / Key Columns	Access Pattern
faces (SQLite)	id INTEGER PK, name TEXT, embedding BLOB (128-dim float32), image_path TEXT	INSERT on registration; SELECT all on recognition (cosine scan); DELETE on identity removal
misclassifications (SQLite)	id INTEGER PK, true_name TEXT, predicted TEXT, score REAL, timestamp TEXT, exported BOOLEAN	INSERT on HITL report; COUNT for threshold check; SELECT for export; DELETE after re-training
airflow metadata (PostgreSQL)	dag_run, task_instance, xcom, connection, variable tables (Airflow-managed schema)	Managed exclusively by Airflow; Flask API never touches this database

Table — Database schemas and access patterns

4 Network Architecture & Inter-Service Communication

All six service groups share a single Docker bridge network named **mlops-net**. Docker's embedded DNS resolver maps each container's service name to its internal IP address, so services reference each other by hostname rather than by IP. The following table documents every inter-service communication path, the protocol used, and the direction of data flow.

Source Service	Target Service	Protocol / Endpoint	Purpose
flask-api	mlflow-server	HTTP GET mlflow:5000/api/2.0/mlflow/...	Load best Production model at startup and after reload notification
flask-api	airflow-webserver	HTTP POST airflow:8080/api/v1/dags/.../dagRuns (JWT auth)	Trigger retraining DAG when misclassification threshold is breached
prometheus	flask-api	HTTP GET flask-api:8080/metrics (scrape)	Collect recognition latency and misclassification Gauge metrics every 15s

Source Service	Target Service	Protocol / Endpoint	Purpose
prometheus	node-exporter	HTTP GET node-exporter:9100/metrics	Collect host CPU/mem/disk metrics every 15s
prometheus	dcgm-exporter	HTTP GET dcgm-exporter:9400/metrics	Collect NVIDIA GPU metrics every 15s
grafana	prometheus	HTTP PromQL queries prometheus:9090	Power Grafana dashboards with time-series data
airflow-scheduler	trainer (DockerOperator)	Docker API → new container launch	Spin up trainer container for train_model DAG task
trainer	mlflow-server	HTTP POST mlflow:5000/api/2.0/mlflow/... (log run, register model)	Log experiment runs and register trained model in Staging
airflow (notify_flask task)	flask-api	HTTP POST flask-api:8080/api/trigger_retrain or /reload	Notify Flask to hot-reload new Production model after promotion

Table — Inter-service communication matrix

5 API Endpoint Definitions — Full I/O Specification

The Flask API (app.py) exposes **16 REST endpoints**. All JSON request/response bodies use Content-Type: application/json. The video feed uses multipart/x-mixed-replace (MJPEG streaming). Each endpoint is documented below with its HTTP method, path, expected inputs, and response schema.

5.1 GET Endpoints

Met hod	Endpoint	Input	Response
GET	/	—	HTML page (index.html) — serves the browser-based frontend UI
GET	/video_feed	—	MJPEG stream; multipart/x-mixed-replace — live annotated video from the camera
GET	/health	—	{"status": "ok", "uptime_seconds": int} — liveness probe for container orchestration
GET	/ready	—	{"status": "ready" "not_ready", "model": str, "model_version": str} — readiness probe; not_ready if model not yet loaded
GET	/metrics	—	Prometheus text/plain exposition format — scraped by Prometheus every 15s

Method	Endpoint	Input	Response
GET	/api/status	—	{"mode": str, "last_result": str, "reg_status": str, "verify_result": str, "model_version": str, "misclassify": bool, "hitl_pending": bool} — full state machine snapshot
GET	/api/model_info	—	{"model_name": str, "model_version": str, "device": str, "threshold": float} — currently loaded model metadata and cosine threshold
GET	/api/people	—	[{"name": str, "samples": int}, ...] — list of all enrolled identities with sample counts
GET	/api/misclassification_stats	—	{"total": int, "unique_pending": int, "threshold": float} — current misclassification counter state

Table — GET endpoint definitions with response schemas

5.2 POST Endpoints

Method	Endpoint	Request Body	Response & Side Effects
POST	/api/register	{"name": str}	{"ok": bool} — arms the video loop for face capture; begins collecting embeddings for the named identity
POST	/api/cancel_register	—	{"ok": bool} — cancels an in-progress registration and discards any partial embeddings captured
POST	/api/delete	{"name": str}	{"ok": bool} — removes the named identity from SQLite embedding store and deletes image files from disk
POST	/api/test_recognition	—	{"ok": bool, "name": str, "score": float} — runs recognition against the last seen face crop; useful for threshold tuning
POST	/api/flag_retrain	{"true_label": str}	{"ok": bool, "queued": bool, "stats": {...}} — logs the corrected label; triggers Airflow DAG if misclassification threshold is exceeded
POST	/api/report_misclassification	{"true_name": str, "predicted": str, "score": float}	{"ok": bool, "stats": {...}, "retraining_triggered": bool} — records full misclassification details; threshold check; optional DAG trigger
POST	/api/trigger_retrain	—	{"ok": bool, "message": str} — manual DAG trigger endpoint; bypasses threshold check; intended for admin/testing use

Table — POST endpoint definitions with request/response schemas

5.3 API Design Rationale

The REST API is designed around **state machine transitions** rather than CRUD operations. The `/api/status` endpoint exposes the full state machine snapshot, allowing the frontend to render context-appropriate UI controls without storing any local state. This means the frontend is **fully stateless** — refreshing the browser or reconnecting from a different client always yields a consistent view of the current system state.

The separation between `/api/flag_retrain` (lightweight threshold check) and `/api/report_misclassification` (full misclassification record) allows clients to report errors at different levels of detail. A simple UI button can call `flag_retrain` with just the corrected label; a more sophisticated client can call `report_misclassification` with the predicted identity and confidence score for richer audit logging.

6 Loose Coupling — Frontend vs Inference Engine

The rubric requirement is that the frontend UI and backend inference engine must be **independent software blocks connected only via configurable REST API calls**. This is fully satisfied at the implementation level as demonstrated below.

Criterion	Implementation Evidence
Separation of concerns	<code>index.html</code> uses <code>fetch()</code> calls exclusively. No shared Python imports, no shared memory, no WebSocket — pure HTTP between frontend and backend.
Configurable API base	<code>MLFLOW_TRACKING_URI</code> , <code>AIRFLOW_BASE_URL</code> , and <code>THRESHOLD</code> are all injected via <code>docker-compose.yaml</code> environment section — zero code changes needed to reconfigure any endpoint.
Frontend independence	<code>index.html</code> is a static file servable from any CDN or static host. The Flask backend at <code>:8080</code> is the sole runtime dependency; no build step or framework coupling.
Backend independence	The inference engine runs headlessly with no UI dependency. <code>/metrics</code> is consumed by Prometheus; <code>/video_feed</code> by any MJPEG-compatible client (browser <code>img</code> tag, VLC, curl).
REST-only coupling	All state transitions (register, delete, <code>flag_retrain</code> , cancel) are mediated by HTTP POST with JSON payloads; all reads by HTTP GET with JSON — no other coupling mechanism exists.

Table — Loose coupling compliance matrix

7 Implementation Quality — Coding Standards

7.1 Python Coding Style (PEP 8)

Criterion	Evidence / Notes
Naming conventions	snake_case for functions/variables, UPPER_SNAKE for constants, PascalCase for classes. Consistent throughout all modules.
Import organisation	Standard library → third-party → local imports. Minor: app.py imports datetime twice (redundant but harmless). No missing <code>__future__</code> imports.
Line length	Most lines within 100 characters. Some <code>bash_command</code> strings in <code>training_pipeline.py</code> contain long heredoc lines — acceptable for shell commands.
Function length / SRP	Functions are generally single-purpose. <code>gen_frames()</code> is longest (~80 lines) but appropriately combines camera I/O and frame annotation — splitting it would create artificial coupling.
Type annotations	Most function signatures lack type hints. Only <code>detect_faces_mp()</code> uses partial annotation. Adding hints would improve IDE support and static analysis with mypy.
Unused code	No significant dead code. Double <code>datetime</code> import in <code>app.py</code> is the only minor redundancy identified.
Module-level guard	All scripts with top-level logic use <code>if __name__ == '__main__':</code> (<code>app.py</code> , <code>sweep_optuna.py</code> , <code>main.py</code> , <code>prepare_data.py</code> , <code>register_model.py</code>).

Table — PEP 8 / coding style evaluation

7.2 Logging

Structured logging is implemented uniformly across all modules using Python's standard logging library. Each module initialises its logger as `log = logging.getLogger(__name__)`, ensuring that log records are tagged with the originating module name. `basicConfig` uses a consistent timestamp format (`%Y-%m-%d %H:%M:%S`) across all services.

Criterion	Evidence
Logger initialisation	Each module: <code>log = logging.getLogger(__name__)</code> . Consistent <code>basicConfig</code> with timestamp format <code>%Y-%m-%d %H:%M:%S</code> .
Log levels used	<code>log.info()</code> for milestones, <code>log.warning()</code> for recoverable issues, <code>log.error(..., exc_info=True)</code> for exceptions — full tracebacks included.
Training pipeline	<code>training_pipeline.py</code> uses module-level log for all DAG task progress, export counts, promotion decisions, and Flask notification results.
Docker streaming	<code>PYTHONUNBUFFERED=1</code> set in all Docker environments — ensures real-time log streaming to Docker stdout without buffering delays.
Prometheus integration	<code>update_misclassification_metric()</code> keeps Gauge in sync with SQLite; <code>_record_recognition()</code> tracks latency for histogram export.

Table — Logging evaluation

7.3 Exception Handling

Exception handling is comprehensive, targeting both infrastructure failures and ML-specific edge cases. The design principle is **fail gracefully at the boundary, propagate with context**: each subsystem catches its own errors, logs with `exc_info=True` for full tracebacks, and returns a safe default or re-raises with a stable interface.

Context	Handling Strategy
Database operations	<code>db_register()</code> and <code>db_recognize()</code> wrap <code>sqlite3.connect()</code> in <code>try/except sqlite3.Error</code> . <code>db_recognize()</code> returns ('Unknown', -1.0) on failure — a safe default that the caller can distinguish from a low-confidence match.
Model loading	<code>load_best_model_from_mlflow()</code> wraps the MLflow call in <code>try/except Exception</code> , logs the traceback, and re-raises. Boot-time catch serves requests without a model rather than crashing the container.
Embedding extraction	<code>get_embedding()</code> raises <code>RuntimeError("Embedding extraction failed")</code> wrapping the original exception, giving callers a stable interface regardless of the underlying PyTorch or MediaPipe failure mode.
Camera reconnection	<code>gen_frames()</code> detects <code>cap.read()</code> failure, logs a warning, releases the capture, and attempts <code>find_camera()</code> reconnection — the stream recovers automatically without user intervention.
Airflow JWT auth	<code>_get_airflow_token()</code> distinguishes <code>ConnectionError</code> from <code>HTTPError</code> . <code>_trigger_retraining_dag()</code> catches all exceptions without crashing the Flask request handler — a failed DAG trigger does not fail the API response.
Data preparation	<code>validate_dataset()</code> raises <code>FileNotFoundError</code> with actionable guidance (tells the user which path is missing). <code>write_manifest()</code> catches <code>OSError</code> and re-raises after logging.
Re-embedding	<code>re_embed_all_faces()</code> wraps each person in <code>try/except</code> with per-person error logging — one corrupt face directory does not abort the re-embedding of all other enrolled identities.
Signal handling	<code>_handle_signal()</code> is registered for <code>SIGINT</code> and <code>SIGTERM</code> ; releases the camera resource and calls <code>os._exit(0)</code> — prevents zombie camera handles on container shutdown.

Table — Exception handling coverage by subsystem

8 Overall Assessment

The Low-Level Design is fully consistent with the High-Level Design. Every architectural decision described in the HLD — three-plane separation, REST-only coupling, MLflow registry as the coordination layer, dual-trigger re-training policy, and post-retrain data lifecycle management — is concretely implemented at the Docker Compose service level and traceable to specific files, classes, and endpoints documented in this document. The codebase adheres to PEP 8, uses structured logging uniformly, and implements comprehensive exception handling covering both infrastructure and ML-specific failure modes. The system is well-positioned for production deployment and ongoing maintenance.

The companion **Test Plan & Test Cases** document provides full coverage of the unit and integration tests that verify the behaviour described in this LLD, including model correctness tests, data pipeline tests, and API endpoint tests.